

INTRODUCTION OF STD::COLONY TO THE STANDARD LIBRARY

I. Introduction

This is an initial draft based on a positive response to functionality and performance of the reference implementation. Colony has at this point been receiving community feedback for the past 1.5 years from Boost, SG14 and individual developers, and on the basis of support from SG14, I am seeking indication of levels of interest for inclusion in the standard. At this stage I would also like to ask for feedback on the existing design/interface, and suggestions prior to proceeding to wording.

Sometimes while programming we come across situations where order is unimportant, but where data is heavily interlinked, iterated over frequently, and changing often. An example would be a central 'entity' class in most video game engines. These are 'has a'-style objects rather than 'is a'-style objects, which reference other shared resources such as sprites, sound fx and so on. Those resources are typically located in separate containers. The 'entities' themselves are in turn referenced by other structures such as quadtrees/octrees (for collision detection), level structures, and the like. When iterating over entities, one has to take into account the fact that they may be removed from the game at any time (for example, a wall gets destroyed and therefore no longer requires processing) and new entities inserted (for example, if a new enemy arrives during a level). While this is happening, the inter-linkages between entities, resource collections and superstructures like levels and quadtrees, must stay valid. The order of the entities and resources themselves within the containers is, typically, unimportant.

There are obviously many different ways of allowing objects to reference each other under C++, but the fastest are pointers, closely followed by indexes (for contiguous containers). Unfortunately the container which has the best iteration performance in the standard library, `vector`^[1], also loses pointer validity to elements within it upon insertion, and also pointer/index validity upon erasure. This leads to sophisticated and restrictive workarounds when developers attempt to utilize `vector` under the above circumstances.

Colony is a specifically unordered-but-sortable templated data container which allows for fast iteration and direct pointer linkage, without losing pointer/iterator validity to non-erased/non-end() elements when insertions or erasures occur. It is its own abstract data type, but is most similar to a "bag", "bucket-array" or "multiset" ADT, the central difference being the absence of key values. On the basis of a fully-developed reference implementation the following performance characteristics have been established:

Specifically, a colony has better overall performance than any standard library container when:

- a. Insertion order is unimportant
- b. Insertions and erasures to the container are occurring frequently in performance-critical code, **and/or**
- c. Pointers/iterators to non-erased/non-end() container elements must not be invalidated by insertion or erasure.

While the benchmarks^[1] referenced in the appendix are a better tool for understanding the performance characteristics, the comparative speed characteristics for most-commonly-used operations in the above scenario as described are:

- *Singular (non-fill), unordered insertion*: better than any std:: library container except some implementations of deque, when compared with insertion at the fastest location (eg. at back for std::vector, front for std::list). This is achieved by using a multiple-memory-block model with a growth factor, which removes a need for reallocation upon reaching capacity. Additionally this model preserves pointer validity upon insertion.
- *Erasure (from random location)*: better or equal to any std:: library container. This assumes the erasure location is found during iteration or via pre-existing pointers/iterators to the element. The result is achieved via the use of a jump-counting skipfield to

indicate erasures, as opposed of a vector/deque's method of reallocating elements after the erased element. It is true regardless of whether a `remove_if` idiom is utilized with a vector in the comparison. Utilizing a skipfield also preserves pointer validity to non-erased elements post-erasure.

- *Iteration*: better than any `std::` library container when pointer validity post erasure/insertion is preserved (eg. map, multiset, list). Where pointer validity is unimportant, better than any `std::` library container except deque and vector. This result is due to the largely contiguous element storage from the multiple-memory-block model in combination with utilization of the jump-counting skipfield, which does not utilize branching code for iteration.

As explored in the benchmarks^[1] there are some vector/deque modifications/workarounds which can outperform colony for iteration while maintaining pointer validity, but which have slower insertion and erasure speeds, and also typically have a cost to usability or memory requirements. When the ratio of insertions/erasures to iterations is greater than 1% insertions/erasures per 3600 full iterations over data, colony will outperform these workarounds. Colony's other advantages are the freeing and recycling of unused memory on-the-fly, and the guaranteed validity of pointers/references to non-erased elements regardless of insertion and erasure (which makes programming with containers of inter-related data structures much simpler). Iterators which do not point to `end()` or erased elements are also guaranteed to remain valid.

II. Motivation and Scope

While the conditions below are common across multiple domains, for the benefit of those unfamiliar with any specific scenarios, I will present the motivation with examples from game development. When working on game engines we are predominantly dealing with collections of data where:

- a. Elements within data collections refer to elements within other data collections (through a variety of methods - indices, pointers, etc). An example is a game entity referring to both a texture object and collision blocks, as well as sound data. These references must stay valid throughout the course of the game/level. For this reason, any container (or use of a container) which causes pointer or index invalidation can create difficulties or necessitate workarounds.
- b. Order is unimportant for the most part. The majority of data collections are simply iterated over, transformed, referred to and utilized with no regard to order.
- c. Erasing or otherwise removing or deactivating objects occurs frequently in-game and in realtime (though often erasures will be implemented to occur at the end of a frame due to multithreading concerns). An example could be destroying a wall, or a game enemy. For this reason methods of erasure which create strong performance penalties are avoided.
- d. Creating new objects and adding them into the gameworld on-the-fly is also common - for example, a tree which drops leaves every so often, or a quadtree.
- e. We don't always know in advance how many elements there will be in a container at the beginning of development, or even at the beginning of a level during playback. Genericized game engines in particular have to adapt to considerably different user requirements and scopes. For this reason extensible containers which can expand and contract in realtime are usually necessary.
- f. For performance reasons, memory storage which is more-or-less contiguous is preferred. Lists, vectors of pointers to dynamically-allocated objects, and maps as implemented in the standard library are unusable.
- g. Memory wastage is avoided, and in particular, any container which allocates upon initialisation tends to be avoided as this can incur purposeless memory and performance costs.

`std::vector`, in it's default state, does not meet these requirements due to:

1. Poor (non-fill) singular insertion performance (regardless of position) due to the need for reallocation upon reaching capacity
2. Poor erasure performance for large amounts of data (even with `remove_if` duration is highly variable)
3. Insert invalidates pointers/iterators to all elements
4. Erase invalidates pointers/iterators/indexes to all elements after the erased element
5. Requires single contiguous memory block (typically larger than cache lines) which increases chance of allocation failures for large amounts of data on insert

To meet these requirements, game developers tend to either (a) develop their own custom containers for given scenarios or (b) develop workarounds for the problems of vector. These workarounds are many and varied, but the most common are probably:

1. Using a boolean flag (or similar) to indicate the inactivity of an object (as opposed to actually erasing from the vector). When erasing, one simply adjusts the boolean flag, and when iterating, items with the adjusted boolean flag are skipped. External elements refer to elements within the container via indexes rather than pointers (which can be invalidated upon insertion).

Advantages: Fast erasure.

Disadvantages: Slow to iterate due to branching.

2. Utilizing a vector of data with a secondary vector of indexes. When erasing, the erasure occurs in the vector of indexes, not the vector of data, and when iterating, one iterates over the vector of indexes, then accessing the data from the vector of data via the index.

Advantages: Faster iteration.

Disadvantages: Erasure still incurs some reallocation cost, can increase jitter.

3. Combining a swap-and-pop mechanism with some form of dereferenced lookup system to enable contiguous element iteration (sometimes known as a 'packed array', various other names: <http://bitsquid.blogspot.ca/2011/09/managing-decoupling-part-4-id-lookup.html>). In this case when erasing we swap the back element of the vector with the element being erased, then pop the (swapped) back element. When iterating over the data we simply iterate through the vector of elements. To maintain valid external references to elements, which may be swapped at any time, we must also maintain a vector of indexes (or similar) and update the index numbers which correspond with the element being erased and the element being moved. In addition external objects referring to elements within the container must store a pointer to the index for the element in question. There are many varied alternatives to this method. The method is more useful when the container's data is mostly being iterated over, with fewer external references to the individual elements.

Advantages: Iteration is at standard vector speed.

Disadvantages: Erase could be slow if objects are large and/or non-trivially copyable, making swap cost thereby large. All references to elements incur additional costs due to the dereferencing system.

All three techniques have the disadvantage of slow singular insertions, and the first two will also continually expand memory usage when erasing and inserting over periods of time. The third deals better with this scenario as it swaps from the back rather than leaving gaps in the elements vector, however it will not free memory if the number of elements expands then contracts. It will also suffer in performance if elements within the container are heavily referred to by external objects/elements in performance-critical situations, or if the elements are large and/or swap/copy is non-trivial in other ways.

Colony is an attempt to bring a more generic solution to this situation. It has the advantage of good iteration speed, a similar erasure speed to the boolean technique described above, and a singular insertion speed which is much faster than a vector's, similar to a good deque implementation's. It never causes pointer invalidation to non-erased elements during erasure or insertion and the memory locations of erased elements are either reused by subsequent insertions or released on-the-fly when the encapsulating memory block becomes empty of elements.

III. Impact On the Standard

This is a pure library addition, no changes are necessary to the standard besides from the introduction of the colony container.

The reference implementation of colony is available here as plf::colony: <http://www.plflib.org/colony.htm>

IV. Design Decisions

The key technical features of a colony are as follows:

- Unordered non-associative data container
- Never invalidates pointers to non-erased elements
- Reuses or frees memory from erased elements (memory is freed if memory block becomes empty, see below)
- In the context of general game data requirements (as described in section III), should be faster than any unmodified standard library container
- Iterators to non-erased elements guaranteed to stay valid unless iterator == end()

The full list of abstract requirements to support these features are as follows:

- Must use multiple memory-blocks as this prevents element reallocation on insertion
- Blocks must be freed once empty

- Blocks must be removable with low performance cost and without pointers to elements being invalidated
- Must have a mechanism for reusing erased element locations upon insertion
- Erased elements are recorded in a skipfield, skipped over during iteration - prevents element reallocation on erasure
- Skipfield design must allow for $O(1)$ ++ and -- iteration

To summarise and simplify we can say there are three necessary aspects to colony to make it function as it does, and which define any implementation:

1. A multiple-memory-block based allocation pattern which allows for the fast removal of memory blocks when they become empty of elements (regardless of their location within the series of blocks), without element pointer invalidation.
2. A skipfield to enable the skipping over of erased elements during iteration, which should not necessitate the use of branching code during iteration for performance reasons.
3. A mechanism for reusing erased element locations upon subsequent insertions.

In the case of the reference implementation I utilized a chained-group allocation pattern (http://www.plfib.org/chained_group_allocation_pattern.htm) for the memory blocks; which is a doubly-linked intrusive list of nodes containing (a) memory blocks, (b) memory block metadata and (c) skipfields. The metadata in question includes information necessary for an iterator to iterate over colony elements, such as the last insertion point within the memory block, and other information useful to specific functions such as the total number of active elements in the node. This 'linked-list'-style pattern removes the possibility of non- $O(1)$ operations when freeing empty memory blocks from the colony structure; as compared to a vector of pointers to memory blocks. A vector of pointers to memory blocks may however enable faster insertions while increasing implementation complexity. Comparative benchmarks would be necessary to establish the overall optimal approach.

For the skipfield a boolean skipfield cannot be used:

- ++ iteration is $O(\text{random})$, iterators require $O(1)$ operations
- Any boolean skipfield creates branching code in the iterator
- Dependent on CPU branch prediction for performance
- Iterator code complexity increases
- Waste of 7 bits (and 254 states) per element if aiming for speed (byte-level addressing- bool or char) rather than storage size (bitfield - slower)

Instead the reference implementation utilizes the jump-counting skipfield pattern (full reference paper available here: http://www.plfib.org/the_jump_counting_skipfield_pattern.pdf), a numeric pattern I developed which allows for $O(1)$ time complexity iterator operations and performs better during iteration due to a lack of branching code. It accomplishes this by storing and modifying (during insertion and erasure) numbers corresponding to the number of elements in a run of erased elements, and simply adding them to the current iterator position during iteration instead of checking their state. This avoids the looping branching code necessary for iteration with, for example, a boolean skipfield.

Iteration performance of vector with boolean skipfield vs colony with boolean skipfield vs colony with jump-counting skipfield (GCC5):

For the erased location re-use mechanism, the reference implementation uses a stripped-down custom internal stack class based on `plf::stack` (<http://www.plfib.org/stack.htm> - outperforms all other `std::` containers in a stack context and across compilers). `plf::stack` uses the same chained-group memory allocation pattern as `plf::colony` ie. it is made up of a linked chain of 'groups', structs containing a memory block and metadata, with a growth factor of 2 for the memory blocks. When capacity of the current memory block is reached, a new group is created and linked to the current memory block.

Time to push all elements + read-and-pop all elements, `plf::stack` vs `std::stack` (`std::deque`) vs `std::vector` (GCC5):

An alternative route of using a "free list" (explanation: <http://bitsquid.blogspot.ca/2011/09/managing-decoupling-part-4-id-lookup.html>) of erased elements for the re-use mechanism has been explored and the following issues identified:

1. A colony element could be smaller in size than a pointer and thus a union with such would dramatically increase the amount of wasted space in circumstances with low numbers of erasures - moreso if the pointer type supplied by the allocator happens to be non-trivial.
2. Given that a free list will jump between colony groups a lot, consolidating a free list after removing empty groups from a colony

would typically result in a very slow operation filled with cache misses. In the context of jitter-sensitive domains such as game development this is unacceptable.

V. Technical Specifications

Colony meets the requirements of the C++ [Container](#), [AllocatorAwareContainer](#), and [ReversibleContainer](#) concepts.

For the most part the syntax and semantics of colony functions are very similar to all std:: c++ libraries. Formal description is as follows:

```
template <class T, class Allocator = std::allocator<T>, typename Skipfield_Type = unsigned short> class colony
```

T - the element type. In general T must meet the requirements of [Erasable](#), [CopyAssignable](#) and [CopyConstructible](#). However, if `emplace` is utilized to insert elements into the colony, and no functions which involve copying or moving are utilized, T is only required to meet the requirements of [Erasable](#). If `move-insert` is utilized instead of `emplace`, T must also meet the requirements of [MoveConstructible](#).

Allocator - an allocator that is used to acquire memory to store the elements. The type must meet the requirements of [Allocator](#). The behavior is undefined if `Allocator::value_type` is not the same as T.

Skipfield_Type - an unsigned integer type. This type is used to create the skipfield and the maximum size of element memory blocks is constrained by it's bit-depth due to the nature of a jump-counting skipfield. For example, `unsigned short` on most platforms is 16-bit and therefore constrains the size of individual memory blocks to a maximum of 65535 elements. `unsigned short` has been found to be the optimal type for performance based on benchmarking. However there may be some memory-constrained situations where element block allocations of more than 255 elements at a time would not be desirable. In these situations, `unsigned char` may be used for the skipfield instead, resulting in some additional memory usage saving for the skipfield itself. It is unlikely for there to be any circumstances which benefit from a skipfield bit-depth greater than `unsigned short`. If `Skipfield_Type` is not an unsigned integer type, behaviour is undefined.

Basic example of usage

```
#include <iostream>
#include "plf_colony.h"

int main(int argc, char **argv)
{
    plf::colony<int> i_colony;

    // Insert 100 ints:
    for (int i = 0; i != 100; ++i)
    {
        i_colony.insert(i);
    }

    // Erase half of them:
    for (plf::colony<int>::iterator it = i_colony.begin(); it != i_colony.end(); ++it)
    {
        it = i_colony.erase(it);
    }

    // Total the remaining ints:
    int total = 0;

    for (plf::colony<int>::iterator it = i_colony.begin(); it != i_colony.end(); ++it)
    {
        total += *it;
    }

    std::cout << "Total: " << total << std::endl;
    std::cin.get();
}
```

```
    return 0;
}
```

Example demonstrating pointer stability

```
#include <iostream>
#include "plf_colony.h"

int main(int argc, char **argv)
{
    plf::colony<int> i_colony;
    plf::colony<int>::iterator it;
    plf::colony<int*> p_colony;
    plf::colony<int*>::iterator p_it;

    // Insert 100 ints to i_colony and pointers to those ints to p_colony:
    for (int i = 0; i != 100; ++i)
    {
        it = i_colony.insert(i);
        p_colony.insert(&(*it));
    }

    // Erase half of the ints:
    for (it = i_colony.begin(); it != i_colony.end(); ++it)
    {
        it = i_colony.erase(it);
    }

    // Erase half of the int pointers:
    for (p_it = p_colony.begin(); p_it != p_colony.end(); ++p_it)
    {
        p_it = p_colony.erase(p_it);
    }

    // Total the remaining ints via the pointer colony (pointers will still be valid even after insertions)
    int total = 0;

    for (p_it = p_colony.begin(); p_it != p_colony.end(); ++p_it)
    {
        total += (*p_it);
    }

    std::cout << "Total: " << total << std::endl;

    if (total == 2500)
    {
        std::cout << "Pointers still valid!" << std::endl;
    }

    std::cin.get();
    return 0;
}
```

Iterator Invalidation

All read-only operations, swap, std::swap	Never
clear, reinitialize, operator =	Always
reserve, shrink_to_fit	Only if capacity is changed
change_group_sizes, change_minimum_group_size, change_maximum_group_size	Only if supplied minimum group size is larger than smallest group in colony, or supplied maximum group size is smaller than largest group in colony.

erase	Only for the erased element
insert, emplace	If an iterator is == end() it may be invalidated by a subsequent insert/emplace in some cases. Otherwise no.

Member types

Member type	Definition
value_type	T
allocator_type	Allocator
size_type	Allocator::size_type (pre-c++11) std::allocator_traits<Allocator>::size_type (post-c++11)
difference_type	Allocator::difference_type (pre-c++11) std::allocator_traits<Allocator>::difference_type (post-c++11)
reference	Allocator::reference (pre-c++11) value_type & (post-c++11)
const_reference	Allocator::const_reference (pre-c++11) const value_type & (post-c++11)
pointer	Allocator::pointer (pre-c++11) value_type & (post-c++11)
const_pointer	Allocator::const_pointer (pre-c++11) std::allocator_traits<Allocator>::const_pointer (post-c++11)
iterator	BidirectionalIterator
const_iterator	Constant BidirectionalIterator
reverse_iterator	BidirectionalIterator
const_reverse_iterator	Constant BidirectionalIterator

Iterators for a colony cannot be random access because any +, -, += and -= operators would be non-O(1), due to the need to iterate over a skipfield. However member overloads for the standard library functions advance(), next(), prev() and distance() are available in the reference implementation. These are significantly faster than O(n) in most cases.

Constructors

default	explicit colony(const allocator_type &alloc = allocator_type())
fill	explicit colony(const size_type n, const unsigned short min_group_size = 8, const unsigned short max_group_size = std::numeric_limits<Skipfield_type>::max(), const allocator_type &alloc = allocator_type()) explicit colony(const size_type n, const value_type &element, const unsigned short min_group_size = 8, const unsigned short max_group_size = std::numeric_limits<Skipfield_type>::max(), const allocator_type &alloc = allocator_type())
range	template<typename InputIterator> colony(const InputIterator &first, const InputIterator &last, const unsigned short min_group_size = 8, const unsigned short max_group_size = std::numeric_limits<Skipfield_type>::max(), const allocator_type &alloc = allocator_type())
copy	colony(const colony &source)
move	colony(colony &&source) noexcept (C++11 and upwards)
initializer list	colony(const std::initializer_list<value_type> &element_list, const unsigned short min_group_size = 8, const unsigned short max_group_size = std::numeric_limits<Skipfield_type>::max(), const allocator_type &alloc = allocator_type())

Member functions

Insert

single element	iterator insert (const value_type &val)
fill	iterator insert (const size_type n, const value_type &val)
range	template <class InputIterator> iterator insert (const InputIterator &first, const InputIterator &last)
move	iterator insert (value_type&& val) (C++11 and upwards)
initializer list	iterator insert (const std::initializer_list<value_type> &il)

- iterator insert(const value_type &element)

Inserts the element supplied to the colony, using the object's copy-constructor. Will insert the element into a previously erased element slot if one exists, otherwise will insert to back of colony. Returns iterator to location of inserted element. Example:

```
plf::colony<unsigned int> i_colony;  
i_colony.insert(23);
```

- iterator insert (const size_type n, const value_type &val)

Inserts n copies of val into the colony. Will insert the element into a previously erased element slot if one exists, otherwise will insert to back of colony. Returns iterator to location of first inserted element. Example:

```
plf::colony<unsigned int> i_colony;  
i_colony.insert(10, 3);
```

- template <class InputIterator> iterator insert (const InputIterator &first, const InputIterator &last)

Inserts a series of value_type elements from an external source into a colony holding the same value_type (eg. int, float, a particular class, etcetera). Stops inserting once it reaches last. Example:

```
// Insert all contents of colony2 into colony1:  
colony1.insert(colony2.begin(), colony2.end());
```

- iterator insert(value_type &&element) **C++11 and upwards**

Moves the element supplied to the colony, using the object's move-constructor. Will insert the element in a previously erased element slot if one exists, otherwise will insert to back of colony. Returns iterator to location of inserted element. Example:

```
std::string string1 = "Some text";  
  
plf::colony<std::string> data_colony;  
data_colony.insert(std::move(string1));
```

- iterator insert (const std::initializer_list<value_type> &il)

Moves the element supplied to the colony, using the object's move-constructor. Will insert the element in a previously erased element slot if one exists, otherwise will insert to back of colony. Returns iterator to location of inserted element. Example:

```
std::initializer_list<int> some_ints = {4, 3, 2, 5};  
  
plf::colony<int> i_colony;  
i_colony.insert(some_ints);
```

Erase

single element	iterator erase(const iterator &it)
range	void erase(const iterator &first, const iterator &last)

- `iterator erase(const iterator &it)`

Removes the element pointed to by the supplied iterator, from the colony. Returns an iterator pointing to the next non-erased element in the colony (or to `end()` if no more elements are available). This must return an iterator because if a colony group becomes entirely empty, it will be removed from the colony, invalidating the existing iterator. Attempting to erase a previously-erased element results in undefined behaviour (this is checked for via an `assert` in debug mode). Example:

```
plf::colony<unsigned int> data_colony(50);
plf::colony<unsigned int>::iterator an_iterator;
an_iterator = data_colony.insert(23);
an_iterator = data_colony.erase(an_iterator);
```

- `void erase(const iterator &first, const iterator &last)`

Erases all contents of a given colony from `first` to the element before the `last` iterator. Example:

```
plf::colony<int> iterator1 = colony1.begin();
colony1.advance(iterator1, 10);
plf::colony<int> iterator2 = colony1.begin();
colony1.advance(iterator2, 20);
colony1.erase(iterator1, iterator2);
```

Other functions

- `iterator emplace(Arguments ...parameters)` **C++11 and upwards**

Constructs new element directly within colony. Will insert the element in a previously erased element slot if one exists, otherwise will insert to back of colony. Returns iterator to location of inserted element. "...parameters" are whatever parameters are required by the object's constructor. Example:

```
class simple_class
{
private:
    int number;
public:
    simple_class(int a_number): number (a_number) {};
};

plf::colony<simple_class> simple_classes;
simple_classes.emplace(45);
```

- `bool empty()`

Returns a boolean indicating whether the colony is currently empty of elements.

Example: `if (object_colony.empty()) return;`

- `size_type size()`

Returns total number of elements currently stored in container.

Example: `std::cout << i_colony.size() << std::endl;`

- `size_type max_size()`

Returns the maximum number of elements that the allocator can store in the container. This is an approximation as it does attempt to measure the memory overhead of the container's internal memory structures. It is not possible to measure the latter because a copy operation may change the number of groups utilized for the same amount of elements, if the maximum or minimum group sizes are different in the source container.

Example: `std::cout << i_colony.max_size() << std::endl;`

- `size_type capacity()`

Returns total number of elements currently able to be stored in container without expansion.

Example: `std::cout << i_colony.capacity() << std::endl;`

- `void shrink_to_fit()`

Reduces container capacity to the amount necessary to store all currently stored elements. If the total number of elements is larger than the maximum group size, the resultant capacity will be equal to $((\text{total_elements} / \text{max_group_size}) + 1) * \text{max_group_size}$

max_group_size (rounding down at division). Invalidates all pointers, iterators and references to elements within the container.

Example: `i_colony.shrink_to_fit();`

- `void reserve(unsigned short reserve_amount)`

Preallocates memory space sufficient to store the number of elements indicated by `reserve_amount`. The maximum size for this number is limited to the maximum group size of the colony and will be truncated if necessary. The default maximum group size is 65535 on the majority of platforms.

Example: `i_colony.reserve(15);`

- `void clear()`

Empties the colony and removes all elements and groups.

Example: `object_colony.clear();`

- `void change_group_sizes(const unsigned short min_group_size, const unsigned short max_group_size)`

Changes the minimum and maximum internal group sizes, in terms of number of elements stored per group. If the colony is not empty and either `min_group_size` is larger than the smallest group in the colony, or `max_group_size` is smaller than the largest group in the colony, the colony will be internally copy-constructed into a new colony which uses the new group sizes, invalidating all pointers/iterators/references.

Example: `object_colony.change_group_sizes(1000, 10000);`

- `void change_minimum_group_size(const unsigned short min_group_size)`

Changes the minimum internal group size only, in terms of minimum number of elements stored per group. If the colony is not empty and `min_group_size` is larger than the smallest group in the colony, the colony will be internally copy-constructed into a new colony which uses the new minimum group size, invalidating all pointers/iterators/references.

Example: `object_colony.change_minimum_group_size(100);`

- `void change_maximum_group_size(const unsigned short min_group_size)`

Changes the maximum internal group size only, in terms of maximum number of elements stored per group. If the colony is not empty and either `max_group_size` is smaller than the largest group in the colony, the colony will be internally copy-constructed into a new colony which uses the new maximum group size, invalidating all pointers/iterators/references.

Example: `object_colony.change_maximum_group_size(1000);`

- `void reinitialize(const unsigned short min_group_size, const unsigned short max_group_size)`

Semantics of function are the same as "clear(); change_group_sizes(min_group_size, max_group_size);", but without the copy-construction code of the `change_group_sizes()` function - this means it can be used with element types which are non-copy-constructible, unlike `change_group_sizes()`.

Example: `object_colony.reinitialize(1000, 10000);`

- `void swap(colony &source)`

Swaps the colony's contents with that of `source`.

Example: `object_colony.swap(other_colony);`

- `friend void swap(colony &A, source &B)`

External friend function, swaps the colony A's contents with that of colony B (assumes both stacks have same element type).

Example: `swap(object_colony, other_colony);`

- `colony & operator = (const colony &source)`

Copy the elements from another colony to this colony, clearing this colony of existing elements first.

Example: `// Manually swap data_colony1 and data_colony2 in C++03`

`data_colony3 = data_colony1;`

`data_colony1 = data_colony2;`

`data_colony2 = data_colony3;`

- `colony & operator = (const colony &&source) C++11 only`

Move the elements from another colony to this colony, clearing this colony of existing elements first. Source colony becomes invalid but can be safely destructed without undefined behaviour.

Example: `// Manually swap data_colony1 and data_colony2 in C++11`

`data_colony3 = std::move(data_colony1);`

`data_colony1 = std::move(data_colony2);`

`data_colony2 = std::move(data_colony3);`

- `bool operator == (const colony &source)`

Compare contents of another colony to this colony. Returns a boolean as to whether they are equal.

Example: `if (object_colony == object_colony2) return;`

- `bool operator != (const colony &source)`

Compare contents of another colony to this colony. Returns a boolean as to whether they are not equal.

Example: `if (object_colony != object_colony2) return;`

- `iterator begin(), iterator end(), const_iterator cbegin(), const_iterator cend()`

Return iterators pointing to, respectively, the first element of the colony and the element one-past the end of the colony (as per standard STL guidelines).

- `reverse_iterator rbegin(), reverse_iterator rend(), const_reverse_iterator cbegin(), const_reverse_iterator cend()`

Return reverse iterators pointing to, respectively, the last element of the colony and the element one-before the first element of the colony (as per standard STL guidelines).

- `iterator get_iterator_from_pointer(const element_pointer_type the_pointer) (slow)`

Getting a pointer from an iterator is simple - simply dereference it then grab the address ie. `&(*the_iterator);`. Getting an iterator from a pointer is typically not so simple. This function enables the user to do exactly that. This is expected to be useful in the use-case where external containers are storing pointers to colony elements instead of iterators (as iterators have 3 times the size of an element pointer) and the program wants to erase the element being pointed to or possibly change the element being pointed to. Converting a pointer to an iterator using this method and then erasing, is about 20% slower on average than erasing when you already have the iterator. This is less dramatic than it sounds, as it is still faster than all `std::` container erasure times except `std::list`, which it is roughly equal to. However this is generally a slower, lookup-based operation. If the lookup doesn't find a non-erased element based on that pointer, it returns `end()`. Otherwise it returns an iterator pointing to the element in question. Example:

```
plf::colony<a_struct> data_colony;
plf::colony<a_struct>::iterator an_iterator;
a_struct struct_instance;
an_iterator = data_colony.insert(struct_instance);
a_struct *struct_pointer = &(*an_iterator);
iterator another_iterator = data_colony.get_iterator_from_pointer(struct_pointer);
if (an_iterator == another_iterator) std::cout << "Iterator is correct" << std::endl;
```

- `size_type get_index_from_iterator(const iterator/const_iterator &the_iterator) (slow)`

While colony is a container with unordered insertion (and is therefore unordered), it still has a (transitory) order which changes frequently upon erasure and insertion. *Temporary* index numbers are therefore obtainable. These can be useful, for example, when creating a save file in a computer game, where certain elements in a container may need to be re-linked to other elements in other container upon reloading the save file. Example:

```
plf::colony<a_struct> data_colony;
plf::colony<a_struct>::iterator an_iterator;
a_struct struct_instance;
data_colony.insert(struct_instance);
data_colony.insert(struct_instance);
an_iterator = data_colony.insert(struct_instance);
unsigned int index = data_colony.get_index_from_iterator(an_iterator);
if (index == 2) std::cout << "Index is correct" << std::endl;
```

- `size_type get_index_from_reverse_iterator(const reverse_iterator/const_reverse_iterator &the_iterator) (slow)`

The same as `get_index_from_iterator`, but for `reverse_iterators` and `const_reverse_iterators`. Index is from front of colony (same as iterator), not back of colony. Example:

```
plf::colony<a_struct> data_colony;
plf::colony<a_struct>::reverse_iterator r_iterator;
a_struct struct_instance;
data_colony.insert(struct_instance);
data_colony.insert(struct_instance);
r_iterator = data_colony.rend();
unsigned int index = data_colony.get_index_from_reverse_iterator(r_iterator);
if (index == 1) std::cout << "Index is correct" << std::endl;
```

- `iterator get_iterator_from_index(const size_type index) (slow)`

As described above, there may be situations where obtaining iterators to specific elements based on an index can be useful, for example, when reloading save files. This function is basically a shorthand to avoid typing `"iterator it = colony.begin(); colony.advance(it, 50);"`. Example:

```
plf::colony<a_struct> data_colony;
plf::colony<a_struct>::iterator an_iterator;
a_struct struct_instance;
data_colony.insert(struct_instance);
data_colony.insert(struct_instance);
iterator an_iterator = data_colony.insert(struct_instance);
iterator another_iterator = data_colony.get_iterator_from_index(2);
if (an_iterator == another_iterator) std::cout << "Iterator is correct" << std::endl;
```

- `template <iterator_type> void advance(iterator_type iterator, distance_type distance)`

Increments/decrements the iterator supplied by the positive or negative amount indicated by *distance*. Speed of incrementation will almost always be faster than using the ++ operator on the iterator for increments greater than 1. In some cases it may approximate O(1). The *iterator_type* can be an *iterator*, *const_iterator*, *reverse_iterator* or *const_reverse_iterator*.

Example: `colony<int>::iterator it = i_colony.begin(); i_colony.advance(it, 20);`

- `template <iterator_type> iterator_type next(const iterator_type &iterator, distance_type distance)`

Creates a copy of the iterator supplied, then increments/decrements this iterator by the positive or negative amount indicated by *distance*.

Example: `colony<int>::iterator it = i_colony.next(i_colony.begin(), 20);`

- `template <iterator_type> iterator_type prev(const iterator_type &iterator, distance_type distance)`

Creates a copy of the iterator supplied, then decrements/increments this iterator by the positive or negative amount indicated by *distance*.

Example: `colony<int>::iterator it2 = i_colony.prev(i_colony.end(), 20);`

- `template <iterator_type> difference_type distance(const iterator_type &first, const iterator_type &last)`

Measures the distance between two iterators, returning the result, which will be negative if the second iterator supplied is before the first iterator supplied in terms of it's location in the colony.

Example: `colony<int>::iterator it = i_colony.next(i_colony.begin(), 20); colony<int>::iterator it2 = i_colony.prev(i_colony.end(), 20); std::cout << "Distance: " << i_colony.distance(it, it2) << std::endl;`

The full implementation details for the main three aspects of the reference implementation are too large to be included in this document, but are available externally below:

Details on the chained-group allocation pattern are here: http://www.plflib.org/chained_group_allocation_pattern.htm.

The 8900-word paper on the jump-counting skipfield pattern is available here:

http://www.plflib.org/the_jump_counting_skipfield_pattern.pdf, and benchmarks versus boolean skipfields are presented here: http://www.plflib.org/skipfield_comparison.htm.

Lastly, `plf::stack` (on which the reference implementation's internal reduced stack implementation is based) is explained in detail here: <http://www.plflib.org/stack.htm>.

Further details on colony's reference implementation can be found here: <http://www.plflib.org/colony.htm>, including a FAQ.

The reference implementation of colony is available to download here: <http://www.plflib.org/colony.htm#download> or via the github repository: https://github.com/mattreecebentley/plf_colony.

VI. Acknowledgements

Thanks to Glen Fernandes and Ion Gaztanaga for restructuring advice, Robert Ramey for documentation advice, various Boost and SG14 members for support, suggestions and bug reports including Sean Middleditch, Patrice Roy and Guy Davidson. And that jerk from Lionhead for annoying me enough to force me to finally implement the jump-counting

skipfield pattern. Lastly, thanks to Jon Blow for initial advice and Mike Acton for some positive influence.

VII. References

plf::colony benchmarks

As the benchmarks currently need to be updated prior to Issaquah to reflect the performance updates to the reference implementation over the past six months, only links can be provided at this stage:

Benchmarks for GCC can be found here: <http://www.plflib.org/colony.htm#benchmarks>, while benchmarks for MSVC can be found here: http://www.plflib.org/colony_benchmark_msvc_results.htm

Cppcon 2016 Talk

An explanatory talk about Colony and the various approaches involved was given at Cppcon 2016 in Bellevue, Seattle:



Note that the benchmarks presented in this lecture do not reflect current colony performance, which has improved considerably since.

The slides for the talk are available [here](#).